Reflex: Distilling Expectimax into a 15M-Parameter Transformer that Plays 2048 in One Forward Pass, on a $10 Budget

Akshat Singh
github.com/a3ro-dev/reflex-2048

Abstract

We train Reflex, a 14.5M-parameter bidirectional transformer that chooses each 2048 move with a single forward pass and no search, by distilling a hybrid expectimax and lookup-table teacher. 2048 is hard for search-free policies because a game lasts 10,000 to 30,000 moves, so a per-move error rate of 10^{-4} still ends most games early, and the states that matter (late-game boards with eight or more large tiles) are rare. Reflex reads the board as 16 tile tokens plus four action tokens that describe each move's afterstate, is trained with cross-entropy (a strictly proper scoring rule) on 360M depth-3 and 100M depth-4 teacher decisions, and runs at 9.1 ms per move on a laptop CPU. The whole pipeline, including teacher data generation, used 4.8 A100-hours and $8.7 of cloud spend. On 10,000 fresh games Reflex averages 164,395 points (median 131,064; 96.6% reach 2048, 30.0% reach 16384, 3.9% reach 32768). That is about a quarter of its depth-4 teacher and 40% of the best published search-free n-tuple agent (412,785); 2048former, a 53M-parameter transformer trained for about 72 GPU-hours, self-reports 605,491. Two negative results may be more useful than the score. Mixing DAgger relabels of the student's own states into 30% of each batch dropped greedy play from 56,652 to 4,188 mean score while agreement with the teacher on held-out boards stayed at 87%. A 1-ply expectimax over the auxiliary value head scores 4,189. Move probabilities are calibrated on teacher boards (ECE 0.006) and overconfident on boards the student reaches itself (ECE 0.27).

1 Introduction

The strongest 2048 programs search: expectimax with endgame lookup tables reaches 690k to 772k mean score and reaches the 32768 tile in 78% to 86% of games, at tens to thousands of moves per second per core [12]. Search-free agents evaluate the four afterstates once and pick the best. For a decade the best of these were n-tuple networks trained by temporal-difference learning [10, 5, 13], culminating in a 268M-weight two-stage network at 412,785 mean score with 30% reaching 32768 [2]. Recently a 53M-parameter transformer distilled from 6-ply expectimax, 2048former, self-reported 605,491 with no search [6], echoing searchless chess at grandmaster level [9].

These results leave a practical question open: how small and how cheap can a search-free transformer be and still play well? Search-free play is attractive because the cost per move is one fixed-size forward pass, the policy is a calibrated distribution rather than an argmax, and the model is easy to batch on a GPU or run on a laptop. We answer under a hard constraint: one A100 rented for under $10 in total, which buys about 5.5 hours for teacher self-play, training, DAgger and evaluation together (we used 4.8).

Contributions.

- **A compact search-free 2048 policy.** Reflex has 14.5M parameters (3.7× smaller than 2048former) and plays at 164,395 mean score with 3.9% reaching 32768 on 10,000 fresh games, at 9.1 ms per move on a laptop CPU (§5). It is well below the published search-free bar; we report it as a budget point, not a record.
- **A DAgger failure, measured.** Relabelling the student's own states and mixing them into training is the textbook fix for compounding errors. Here it collapsed greedy play by 93% while every offline metric we logged stayed flat (§5.1).
- **Calibration on the student's own states.** Cross-entropy against teacher labels gives calibrated move probabilities on teacher boards (ECE 0.006 at depth 4); on a clean set of boards the final student reaches itself, it agrees with the teacher 61% of the time at a mean confidence of 0.88 (ECE 0.27) (§5.2).
- **Open artifacts**: code, the patched teacher with a board-labelling mode, data checks, training logs and the checkpoint.

2 Related work

Search-free agents. N-tuple networks learned by TD(0) on afterstates [10] were improved by multi-stage weight promotion, temporal coherence and carousel shaping [5, 13], and by optimistic initialisation with tile-downgrading [2], which holds the best published 1-ply score. These tables have 10^8 to 10^9 weights. Transformers distilled from search are the newer line: in chess, a 270M-parameter model trained on Stockfish action values plays at grandmaster level without search [9]; in 2048, 2048former [6] distils 6-ply expectimax into 53M parameters. We follow the distillation line at a quarter of the size and a small fraction of the compute.

Search with learned or hand-built evaluation. macroxue/2048-ai combines expectimax with dynamic-programming tables over the six largest tiles [12]; we use it as the teacher at depths 3 and 4 (published 493k and 626k mean with the large tables). n-tuple networks also gain 150k to 200k points from 3- to 6-ply expectimax [2, 5].

Imitation, distribution shift and calibration. Behaviour cloning suffers compounding errors because the student visits states the expert never shows it; DAgger relabels the student's own states with the expert to bound this [8]. Knowledge distillation transfers a teacher's behaviour into a smaller network [4]. Cross-entropy and the Brier score are strictly proper scoring rules [1], so minimising them against teacher labels drives predicted move probabilities toward the teacher's conditional move frequencies; we report top-label ECE [3] and classwise ECE [7].

3 Method

Game and teacher. Boards are 4×4 grids of tile exponents $e \in \{0, \dots, 17\}$ (0 empty, e meaning 2^e); after each move a 2 (probability 0.9) or a 4 spawns on a uniformly chosen empty cell. The teacher is macroxue/2048-ai with its large lookup tables [12], patched to write every decision it makes in self-play (board, move, valuation, score, seed; 32 bytes per record) and to label arbitrary boards from a file (`-B`). Every teacher transition we checked replays exactly in our batched GPU implementation of the game (moves, spawns and rewards; 35.1M of 35.1M transitions in the files we audited).

Input: tile tokens and afterstate action tokens. Reflex embeds each of the 16 cells as a learned tile-exponent embedding plus learned row and column embeddings. It adds four action tokens, one per move, built from that move's deterministic afterstate: legality, log merge reward, empties, maximum tile, whether the maximum sits in a corner, row and column monotonicity, smoothness, available merges, tile sum, and the log reward of each follow-up move (14 features, an MLP to d). A value token makes 21 tokens in total.

Architecture. Eight pre-norm transformer blocks with bidirectional attention ($d = 384$, 12 heads, MLP ratio 4, GELU) mix the 21 tokens [11]. Each action token emits one logit; illegal moves are masked to $-\infty$, so the softmax is always a distribution over legal moves. The value token regresses the log return-to-go as an auxiliary task; the policy does not use it (we test a 1-ply search over it in §5).

Table 1: 2048 agents on the standard game (90% twos, no undo). Search-free rows evaluate one function per move. Our numbers are on 10,000 fresh games; others are as reported.

Agent	Params	Search	Mean score	32768	Source
macroxue expectimax, large tables	tables	depth 8	711,769	80.5%	[12]
macroxue, large tables (our teacher)	tables	depth 4	625,898	70.7%	[12]
macroxue, large tables (our teacher)	tables	depth 3	493,058	52.5%	[12]
2048former v0.1	53.2M	none	605,491	66.1%	[6]†
2-stage 8×6-tuple OTD+TC, downgrading	268.4M	none	412,785	30.2%	[2]
16-stage 5×6-tuple MS-TC	1342.2M	none	324,710	19%	[5]
Reflex (ours)	**14.5M**	none	**164,395**	**3.9%**	this work
Reflex + 1-ply, value head	14.5M	1-ply	4,189	0%	this work‡

† self-reported on a model card, 2048 games. ‡ 128 games.

Objective. For board s with teacher move a^* and return-to-go G,

$$\mathcal{L} = -\log \pi_\theta(a^* \mid s) \; + \; 0.25\,\mathrm{Huber}\big(v_\theta(s),\, \log(1+G)/10\big), \tag{1}$$

with the value term masked where G is unknown (DAgger labels and unfinished games). Each sample gets a random element of the dihedral group D_4 applied to the board, with the move relabelled to match ($8\times$ augmentation).

DAgger with death tails. Every round the latest checkpoint plays 512 games greedily on the GPU. We keep 2% of visited boards at random and every board from the last 150 moves before each game ends, because a search-free policy usually loses a game through one bad move shortly before the end. The teacher labels these boards at depth 4 in parallel CPU workers, and the trainer mixes the labels in at a fixed share of each batch. We used this in our first run only; §5.1 explains why the final model is trained without it.

4 Experimental setup

Data. Teacher self-play produced 360M depth-3 decisions (21.9k games; mean 455,256, median 387,224, 45.5% reach 32768) and 100M depth-4 decisions (the 1,088 depth-4 games finished at our last count averaged 610,452, 67.3% reaching 32768). DAgger produced 465,690 labelled student states over five rounds. Whole games whose seed is divisible by 50 are held out and never trained on. For DAgger, every 50th labelled state is held out; neighbouring boards of one game then land on both sides of the split, so we also build a clean set: 200,000 states that the final checkpoint reaches in 512 games on fresh seeds, labelled at depth 4 and never trained on.

Training. AdamW ($\beta = (0.9, 0.98)$, weight decay 0.05), batch 8192, 1000 warm-up steps, then a cosine decay to 3% of the peak over a fixed wall-clock budget, in bfloat16 on one A100 80GB. The first run started from scratch at peak lr 8×10^{-4} with batches mixing depth-3 data, depth-4 data and DAgger labels at 40/30/30% (DAgger labels first reached the trainer at step 6000). It collapsed (§5.1). The final model starts from that run's step-6000 checkpoint (49.2M samples, before any DAgger label) and trains 25,915 more steps (212.3M samples) at peak lr 5×10^{-4} on depth-3 and depth-4 data at 50/50 with a 1.9-hour schedule: 261.4M samples and about 2.4 A100-hours in total. On an idle A100 a step runs at 38k samples per second at batch 8192 or 16384, so training is compute-bound; with the evaluator sharing the GPU we saw 29k–37k.

Evaluation. Pure greedy policy (argmax of π_θ, one forward pass per move, no search, no lookahead over spawns) on 10,000 games from seeds disjoint from training, using the end-of-schedule checkpoint (no selection by evaluation score). We report mean, median and maximum score and the rates of reaching 2048 to 65536. Our evaluator did not log per-game scores, so we give no standard error; a second evaluation of the same checkpoint on 2,048 games from other seeds shows the spread instead.

Table 2: Reflex on 10,000 fresh games (greedy, end-of-schedule checkpoint), and the same checkpoint on 2,048 games from other seeds.

Games	Mean	Median	Max	2048	4096	8192	16384	32768
10,000	164,395	131,064	835,640	96.6%	86.4%	63.3%	30.0%	3.9%
2,048	166,767	131,028	796,016	97.1%	86.7%	63.8%	29.7%	4.6%

5 Results

Score. Table 2 gives the final numbers; the two evaluations agree within 1.5% on the mean. Reflex almost always builds 2048 and reaches 16384 in three games out of ten, but rarely 32768 and never 65536 in 10,000 games. That is about 27% of its depth-4 teacher's self-play mean (610,452), 40% of the best published n-tuple agent and 27% of 2048former's self-report (Table 1). Held-out agreement with the depth-4 teacher is 93.6%, so the gap is the compounding of rare errors over games that last 6,476 moves on average.

Learning curve. Watch-mode evaluations of the final run (512 or 256 games, seed 0) rose monotonically: 70,817 mean after 1000 steps, 90,358 after 5000, 121,679 after 14,000 and 157,321 after 20,000. We stopped at the budget, not at a plateau.

Speed. On a laptop CPU (Intel, 8 threads, PyTorch 2.11, batch 1) a move takes 9.1 ms median and 11.0 ms at the 90th percentile, including feature computation. Batched on the A100 the evaluator plays 65k moves per second across 10,000 games.

1-ply over the value head does not work. Choosing $\arg\max_a \; r_a + \mathbb{E}_{s'}[V(s')]$, with the expectation exact over every spawn, scores 4,189 on 128 games, no better than the collapsed policy of §5.1. The value head is not useless: on 50,000 held-out depth-3 boards its prediction correlates at 0.88 with the realised return in log space. But it learned the teacher's return over a whole game, and it cannot separate the four afterstates of one move, whose values differ by far less than its error. A value head meant for search has to be trained for that, for example on action values [9].

5.1 DAgger collapsed greedy play

In the first run, greedy play improved from 17,251 mean score at step 1000 to 45,335 at step 4000 and 56,652 at step 6000 (512 games each). At step 6000 the trainer's periodic data refresh loaded the first 465,690 DAgger labels at a 30% batch share. By step 7500 the same evaluation scored 4,188, and the student's own DAgger rollouts fell from 51,431 to 4,227. Over the same steps held-out agreement with the teacher barely moved: 85.3% to 84.1% at depth 3 and 86.8% to 85.9% at depth 4. The collapsed policy is close to uniform early in the game (0.41/0.09/0.28/0.22 at move 50 of one traced game) and never settles into a corner; its agreement on early-game teacher boards (maximum tile at most 1024) falls from about 84% to 74% while late-game agreement is unchanged. Training on from the collapsed checkpoint with DAgger for 4,000 more steps did not recover it (4,705). We ruled out the obvious bugs: labels from the board-labelling mode match the teacher's own self-play moves on 99.95% of 2,000 depth-4 boards, boards survive the round trip through the teacher exactly, and the step-6000 checkpoint still scores 62,853 on 64 games with the code that evaluated the collapsed one. We did not isolate the cause. Candidates are the batch composition (about 90% of DAgger boards come from the 150 moves before a weak student's death, so 30% of every batch became losing positions), the high learning rate at the moment the data changed, or an interaction between imitation on the student's states and greedy play that our offline metrics cannot see. The practical lesson is that held-out agreement and calibration are not proxies for play in 2048: the only metric that caught this was playing games.

5.2 Calibration on teacher states and on the student's own states

On boards from the teacher's own games the final checkpoint is close to perfectly calibrated (Table 3): mean confidence 0.931 against 93.6% agreement at depth 4, with agreement rising from 89% before the 2048 tile to 95% at 32768 and above. On boards the student reaches itself, confidence barely

Table 3: Calibration of the final checkpoint against teacher labels (15 bins). Depth-3, depth-4: held-out whole games. DAgger (leaky): the every-50th split of first-run DAgger labels, from early students. Student (clean): 200k states the final checkpoint reaches on fresh seeds, labelled at depth 4, never trained on.

Boards	n	Agreement	Mean conf.	ECE	Classwise ECE	NLL
Depth-3 teacher	200k	91.1%	0.921	0.010	0.006	0.205
Depth-4 teacher	200k	93.6%	0.931	0.006	0.004	0.155
DAgger (leaky split)	23.6k	44.5%	0.660	0.216	0.116	2.246
Student, clean	200k	61.0%	0.881	0.271	0.140	3.205

drops (0.881) but agreement falls to 61%, so ECE is 0.27, more than 40 times worse. Agreement on its own boards is lowest early in the game (43% before 2048, 69% at 16384). This is the distribution shift DAgger is meant to fix, and it is large: the student drifts into positions the teacher would play differently, and nothing in its probabilities warns of it.

6 Limitations

- One training lineage, no seeds or ablations: the budget paid for one run and a restart. Effects of the afterstate tokens, DAgger share and model size are untested, and we did not isolate why DAgger collapsed play.
- No standard errors: per-game scores were not logged. Two evaluations on disjoint seeds agree within 1.5%.
- The depth-4 teacher statistics come from the games finished at our last count, not from all 100M depth-4 decisions.
- The teacher is depth 3 and 4. 2048former used depth 6 and about 30× our training compute (72 against 2.4 GPU-hours), so the gap to it mixes model size, teacher strength and compute.
- Labels are hard teacher moves. Soft action-value targets preserve how much worse the other moves are and help in chess [9]; the teacher can export them, but we did not.
- Calibration is measured against the teacher's choices, not against game outcomes. A calibrated probability that the teacher picks a move is not a calibrated probability that the move is safe.

7 Conclusion

A 14.5M-parameter transformer distilled from depth-3 and depth-4 expectimax for under $10 plays 2048 at 164k mean score with one forward pass per move: a long way from the search-free records, but it runs on a laptop. The more durable findings are negative: the student's own boards are where its calibrated-looking probabilities stop meaning anything, and the standard remedy, DAgger, made play far worse while every offline metric stayed flat.

Acknowledgements. The RunPod credits that paid for all compute in this work were given by pranavram98. We thank him for them.

Reproducibility. Code, launcher, data checks, logs and the checkpoint are at `https://github.com/a3ro-dev/reflex-2048`. The teacher patch modifies GPL-3.0 code and is distributed as a patch script.

References

[1] Tilmann Gneiting and Adrian E Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, March 2007. ISSN 1537-274X. doi: 10.1198/016214506000001437. URL `http://dx.doi.org/10.1198/016214506000001437`.

[2] Hung Guei, Lung-Pin Chen, and I-Chen Wu. Optimistic temporal difference learning for 2048. 2021. doi: 10.48550/ARXIV.2111.11090. URL `https://arxiv.org/abs/2111.11090`.

[3] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks, 2017. URL `https://arxiv.org/abs/1706.04599`.

[4] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015. URL https://arxiv.org/abs/1503.02531.

[5] Wojciech Jaśkowski. Mastering 2048 with delayed temporal coherence learning, multi-stage weight promotion, redundant encoding and carousel shaping, 2016. URL https://arxiv.org/abs/1604.05085.

[6] Jacob Keisling. 2048former v0.1: a 50m-parameter encoder-only transformer for 2048. Hugging Face model card, https://huggingface.co/jkeisling/2048former-50m-v0.1, 2025. Self-reported results; accessed 2026-10-23.

[7] Meelis Kull, Miquel Perello-Nieto, Markus Kängsepp, Telmo Silva Filho, Hao Song, and Peter Flach. Beyond temperature scaling: Obtaining well-calibrated multiclass probabilities with dirichlet calibration, 2019. URL https://arxiv.org/abs/1910.12656.

[8] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning, 2010. URL https://arxiv.org/abs/1011.0686.

[9] Anian Ruoss, Grégoire Delétang, Sourabh Medapati, Jordi Grau-Moya, Li Kevin Wenliang, Elliot Catt, John Reid, Cannada A. Lewis, Joel Veness, and Tim Genewein. Amortized planning with large-scale transformers: A case study on chess, 2024. URL https://arxiv.org/abs/2402.04494.

[10] Marcin Szubert and Wojciech Jaskowski. Temporal difference learning of n-tuple networks for the game 2048. In *2014 IEEE Conference on Computational Intelligence and Games*, page 1–8. IEEE, August 2014. doi: 10.1109/cig.2014.6932907. URL http://dx.doi.org/10.1109/CIG.2014.6932907.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017. URL https://arxiv.org/abs/1706.03762.

[12] Hanhong Xue. macroxue/2048-ai: expectimax search with dynamic-programming lookup tables. https://github.com/macroxue/2048-ai, 2022. GPL-3.0; accessed 2026-10-23.

[13] Kun-Hao Yeh, I-Chen Wu, Chu-Hsuan Hsueh, Chia-Chuan Chang, Chao-Chin Liang, and Han Chiang. Multi-stage temporal difference learning for 2048-like games, 2016. URL https://arxiv.org/abs/1606.07374.